

# Bluetooth LE MIDI bridge installation

Fitting an ESP32-C3 SuperMini inside the keyboard so it carries MIDI over Bluetooth alongside its existing USB port, powered from the keyboard's own USB rail. No second cable, no battery.

| LINK                | SUPPLY            | SOLDER JOINTS | ADVERTISES AS |
|---------------------|-------------------|---------------|---------------|
| USART1 · 31250 baud | +5 V, ~120 mA avg | 4             | MPK mini Open |

---

## Before you start

---

The keyboard must already be running the open firmware from `firmware/`. The stock AKAI firmware never drives USART1, so the bridge would sit there advertising and never see a note. The open firmware mirrors every outgoing USB-MIDI event to PA9 and feeds anything arriving on PA10 into the same handler USB SysEx uses.

▲ **PINOUT** Re-verify PA9 and PA10 against the STM32F102R8 datasheet before soldering.

This project's notes flag that the LQFP64 pin table was carried over from an earlier F103 assumption and never independently re-checked. These two pins have no header or test-point access on the AD07 board — they are direct 0.5 mm-pitch chip-pin solders, and the wrong pin is not a cheap mistake. Buzz out continuity with power removed first.

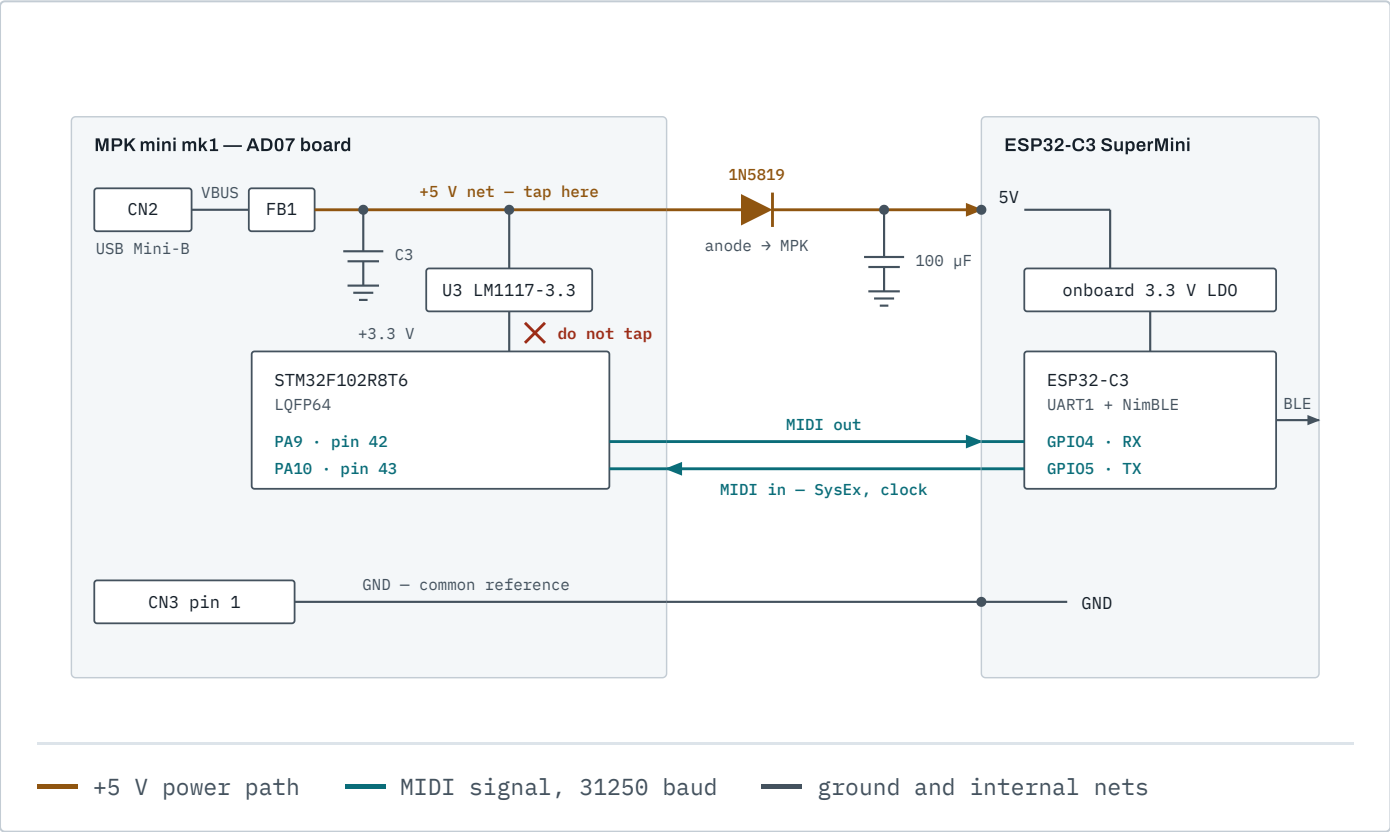
▲ **SUPPLY** Feed the SuperMini's `5V` pin. Never its `3V3` pin.

The keyboard's 3.3 V rail is the output of U3, an LM1117-3.3 in a small SMD package inside a sealed plastic enclosure. Hanging the ESP32-C3's ~120 mA average and ~350 mA transmit peaks off it drops 1.7 V across that regulator and adds up to half a watt of heat it has no thermal budget for. Taking +5 V instead uses the SuperMini's own regulator and leaves the LM1117's load exactly as it was.

The firmware already declares the USB 2.0 high-power maximum (`bMaxPower` 500 mA, against stock's 100 mA), so the combined draw is within what the keyboard is entitled to request. Plug it into a real host port or a powered hub — a passive hub splitting one port between several devices may not deliver it.

| PART                   | SPEC                         | JOB                                                        |
|------------------------|------------------------------|------------------------------------------------------------|
| ESP32-C3 SuperMini     | HW-466AB, USB-C              | The bridge itself                                          |
| Schottky diode         | 1N5819 or any 1 A Schottky   | Stops the SuperMini's USB-C VBUS back-feeding the keyboard |
| Electrolytic capacitor | 100 $\mu$ F, 10 V or better  | Absorbs the radio's current bursts                         |
| Ceramic capacitor      | 100 nF                       | Optional, in parallel with the above                       |
| Hookup wire            | 30 AWG enamelled or silicone | Fine enough for the two chip pins                          |

How it connects



Four wires leave the keyboard: +5 V through the diode, two signal lines that cross TX to RX, and a shared ground. The +5 V tap sits on the same net as C3 and the input of U3 — filtered by FB1, but upstream of the regulator.

# Connections

SIGNAL — BOTH SIDES ARE 3.3 V LOGIC ON THE SAME BOARD, SO THIS IS A DIRECT TTL LINK. NO OPTO-ISOLATOR, NO LEVEL SHIFTING, NO CURRENT-LOOP RESISTORS.

| MPK MINI      | ACCESS                                | ESP32-C3 | CARRIES                       |
|---------------|---------------------------------------|----------|-------------------------------|
| PA9 · pin 42  | Chip pin, 0.5 mm pitch                | GPIO4    | Everything the keyboard plays |
| PA10 · pin 43 | Chip pin, 0.5 mm pitch                | GPIO5    | Editor SysEx and MIDI clock   |
| GND           | CN3 pin 1, a real through-hole header | GND      | Common reference              |

POWER — ON THE AD07 SCHEMATIC, CN2 FEEDS VBUS THROUGH FERRITE FB1 INTO THE +5 V NET, WHICH SUPPLIES C3, C4 AND THE INPUT OF U3

| MPK MINI                                            | IN BETWEEN                                             | ESP32-C3 |
|-----------------------------------------------------|--------------------------------------------------------|----------|
| +5 V net<br>positive leg of C3, or the U3 input pin | 1N5819<br>anode to the MPK, striped cathode to the ESP | 5V       |
| GND                                                 | direct                                                 | GND      |

Put the 100 μF capacitor across the SuperMini's 5V and GND pins, close to the board and the right way round, with the 100 nF in parallel if you have one. The radio draws in bursts and the run back to the keyboard's bulk cap is long enough to matter. The diode's ~0.35 V drop leaves about 4.65 V at the pin, comfortably above the onboard regulator's dropout.

**Finding the pins on the SuperMini.** With the USB-C connector at the bottom and the antenna at the top, the usual HW-466AB layout puts GPIO4 on the left row and GPIO5 at the top of the right row, with 5V and GND adjacent at the bottom of the right row. Board revisions differ — follow the silkscreen, not this description.

## Bring-up

ORDER MATTERS

Each phase is verifiable on its own, and the power tap is deliberately last — it is the one step that is awkward to undo, so the signal path gets proven on separate supplies first. Ticks are saved in this browser.

00

☐ **Verify the pinout**

Open the STM32F102R8 datasheet at the LQFP64 pin table and confirm PA9 and PA10 really are pins 42 and 43. With the keyboard unplugged, buzz continuity from each intended pin to wherever you plan to land the wire.

DONE WHEN

You have seen both pins in the datasheet and confirmed them on the board with a meter.

01

☐ **Flash the bridge on its own**

Nothing connected but the SuperMini's USB-C.

```
# with ESP-IDF activated (built against v6.0.3)
cd esp32-c3-ble-midi
idf.py set-target esp32c3
idf.py build
idf.py -p /dev/ttyACM0 flash monitor
```

`ls /dev/ttyACM*` shows the native USB serial/JTAG port. If none appears, hold B00T, tap RST, release B00T and flash again. Leave the monitor with `Ctrl-C`.

**EXPECT**

The monitor prints `advertising as MPK mini Open`, and the onboard LED blinks once per second.

02

☐ **Pair it with nothing attached**

Prove the Bluetooth half works before any wire touches the keyboard. Pair from a BLE-MIDI-aware path, not the operating system's generic Bluetooth audio screen (see Pairing).

**EXPECT**

The host opens a MIDI port and the LED goes solid. No MIDI will flow yet — there is nothing to send.

03

☐ **Bench-wire the signals only**

Power each board from its own USB port and **leave both 5 V rails separate**. Join GND, then PA9 → GPIO4 and PA10 → GPIO5.

**EXPECT**

A key press arrives at the BLE host. SysEx sent from an editor over BLE gets an answer back from the keyboard.

04

☐ **Fit the power tap**

Only now solder the diode, the capacitor and the +5 V wire, then unplug the SuperMini's USB-C entirely.

**EXPECT**

The bridge boots, advertises and connects on keyboard power alone.

## 05 ☐ Confirm USB still works alongside it

With the bridge connected over Bluetooth, check the keyboard still enumerates and plays over its own USB port. Both transports are meant to run at once.

### EXPECT

The same notes arrive on both the USB MIDI port and the Bluetooth one.

Whenever you reflash the SuperMini over USB-C afterwards, unplug the keyboard first, so the two 5 V sources are not both live on the same node.

## Pairing

ADVERTISES AS "MPK MINI OPEN"

### Linux — BlueZ

Confirmed on BlueZ 5.87: it carries the `midi` profile, and its built-in UUIDs match this firmware's byte for byte, so it is recognised as a MIDI peripheral rather than a generic GATT device.

```
bluetoothctl
# scan on, wait for the name, then
pair    XX:XX:XX:XX:XX:XX
trust   XX:XX:XX:XX:XX:XX
connect XX:XX:XX:XX:XX:XX
```

The `midi` plugin then exposes an ALSA sequencer port — `aconnect -l` lists it. `trust` makes it reconnect by itself.

### macOS & iOS

Audio MIDI Setup → MIDI Studio → Bluetooth, or the Bluetooth panel inside the DAW. Not the system Bluetooth preference pane.

### Windows

BLE MIDI is reachable only through the WinRT MIDI API, which most DAWs do not use. Bridge it with MIDIberry plus loopMIDI, or a vendor BLE-MIDI driver.

Bonds are kept in NVS, so pairing survives the power cycle that happens every time the keyboard is unplugged.

## Reading the LED

ONBOARD LED, GPIO8, ACTIVE LOW — SET STATUS\_LED\_ENABLE TO 0 IN MAIN.C FOR A BOARD THAT WIRES IT ELSEWHERE

| PATTERN              | STATE                                                       |
|----------------------|-------------------------------------------------------------|
| One blink per second | Advertising, no host connected                              |
| Double blink         | Connected, but the host has not subscribed to notifications |
| Solid                | Connected and subscribed — MIDI will flow                   |

## If something is wrong

| SYMPTOM                             | MOST LIKELY CAUSE                                                                                          |
|-------------------------------------|------------------------------------------------------------------------------------------------------------|
| LED blinks, host never sees it      | Paired through the generic Bluetooth screen instead of a BLE-MIDI path                                     |
| Connects, but no notes arrive       | LED double-blinking: the host connected without subscribing. Or PA9 is on the wrong pin — back to phase 00 |
| Notes arrive, editor SysEx does not | The PA10 direction only. Check that wire and that GPIO5 is the ESP's TX                                    |
| Bridge reboots when you play hard   | Missing or too-distant 100 $\mu$ F capacitor, or the diode is undersized                                   |
| Nothing at all on keyboard power    | Diode backwards — striped cathode goes to the ESP                                                          |
| Keyboard itself stops enumerating   | Tapped the 3.3 V rail instead of +5 V, or the host port will not supply 500 mA                             |

## What the bridge does and doesn't carry

- Channel messages, running status, MIDI realtime and fragmented SysEx cross in both directions. Running status is expanded to full messages on the BLE side.
- Outbound MIDI is timestamped as it is parsed and packed so several messages share one notification. Under controller backpressure the sender waits and retries rather than discarding, so a fast knob sweep cannot swallow the Note Off behind it.
- Channel messages sent *to* the keyboard reach the STM32, which ignores everything but SysEx and MIDI realtime — the same as over USB. There is no sound engine and no MIDI-thru. What Bluetooth does gain you is the editor's program dump and restore, and the arpeggiator's external clock.
- One host at a time. SysEx from the keyboard is capped at 256 bytes, against the editor's 101-byte program records.
- The bridge asks for a 7.5–15 ms connection interval, but the host decides. One that insists on a long interval adds latency the bridge cannot do anything about.

The MIDI translation is covered by host-side round-trip tests — `make check` in `esp32-c3-ble-midi/test`. That covers the protocol, not the wiring, the power tap or the UART link, which is what the bring-up order above is for.

---

Service & characteristic UUIDs    03B80E5A-EDE8-4B33-A751-6CE34EC4C700  
7772E5DB-3868-4112-A1A9-F2669D106BF3